RocketRota security audit

Code hosted Supabase authentication and payments

Prepared for Jack McConnell by Codex. Review date 30 October 2026. Internal report containing security-sensitive architecture details, but no credential values or customer records.

Baseline: D:/Projects/Shyft at commit 9a44dda6c5136b492c742672a8a03df2141841a8, including the existing uncommitted working changes. The local checkout and deployed sites were assessed separately; the exact deployed source revision was not attested.

Official documentation was checked on 30 October 2026. Installed source and the lockfile are authoritative for the reviewed versions: TanStack Start 1.169.3, Better Auth 1.7.12, Stripe Node 20.4.1 and application Zod 4.6.7. Source evidence references resolve to the reviewed working tree; future edits can move their line numbers.

Executive assessment

Hold public launch until the high-priority findings are resolved or explicitly bounded and retested. Four high findings have concrete code or configuration evidence: an unencrypted local connection to hosted Supabase, overly broad access to NFC AES keys, password resets that retain sessions, and arbitrary push endpoints reaching the server request layer. A fifth high finding is conditional on an unclaimed admin bootstrap address remaining enabled in production.

The review records 16 findings: 5 high including the conditional bootstrap issue, 9 medium and 2 low. No critical compromise was demonstrated. This is a broad evidence-based audit, not a claim that authentication, payments or the complete system is fully secure.

There is meaningful protection already in place. All 55 hosted public application tables have RLS; private schemas deny browser access; signed payment webhooks and server-side billing ownership checks exist; live main and admin responses have CSP and private caching. Fresh main and admin builds passed. Scans of 382 browser artifact files found no configured private environment values, and selected main plus admin tests passed 93 cases.

First actions

- Restrict browser access to NFC key material and retest workplace manager and supervisor access.
- Require verified database TLS, separate development from production credentials, and confirm the deployed connection behavior.
- Revoke sessions on password reset and require authoritative checks for sensitive operations.
- Restrict push destinations, set outbound timeouts and quotas, and close admin bootstrap after provisioning.

Findings register

Confirmed means the stated code or configuration behavior was observed; it does not mean every consequence was exploited. Conditional and unverified deployment items are explicitly labelled in each finding. High findings should be resolved before exposing real workforce data to a wider launch.

ID	Severity	Finding
S01	High	Database connection does not use TLS
S02	High	Workplace roles can reach NFC cryptographic keys through the Data API
S03	High	Password reset leaves existing sessions valid
S04	High	Push subscription endpoints permit server side requests to arbitrary hosts
S05	High	Admin bootstrap can grant ownership without proving mailbox ownership
S06	Medium	Cookie caching delays session revocation checks
S07	Medium	Legacy main app administration bypasses dedicated admin revocation
S08	Medium	Password recovery wrappers bypass auth router rate limiting
S09	Medium	Passkey login redirects to an unchecked destination
S10	Medium	CSV exports preserve spreadsheet formulas in text fields
S11	Medium	Development shares hosted credentials and uses broad privileges
S12	Medium	Subscription risk events and event ordering need stronger handling
S13	Medium	Dependency audit is no longer clean
S14	Medium	Hosted billing reconciliation rejection returns a server error
S15	Low	Database API grants exceed the server first design
S16	Low	Global headers replace the stricter privacy download policy

Scope and method

The audit combined repository-wide searches, targeted manual data-flow tracing, dependency advisories, fresh production builds, secret-value scanning, mocked security tests, read-only hosted database metadata and small unauthenticated HTTP rejection checks. It covered main and admin TanStack Start entry points, Better Auth, role and workplace checks, billing and station purchases, NFC, push delivery, privacy exports, payroll CSVs, service-worker caching and deployment configuration visible from this workspace.

Hosted project: Shyft, txcleqsbnblbkuihndyc, eu-west-1, PostgreSQL 17.6.1.084. The connected organization reports the Free plan. All 83 local migration versions match the 83 hosted versions. No storage buckets or Edge Functions were listed. Customer records, real NFC keys and real session tokens were not retrieved. No production schema, application data, deployment, payment or secret was changed.

Detailed findings

S01 Database connection does not use TLS

Assessment High severity. Confirmed local configuration against hosted database.

Evidence All three database pool factories supply only DATABASE_URL. Neither inspected environment file requests TLS. A read-only connection using .env.local to the hosted Supabase pooler returned current_user postgres and pg_stat_ssl.ssl false.

Impact Employee records, session values and query results travel from this client to the hosted pooler without transport encryption. This is a demonstrated local-to-hosted exposure; the Coolify application's connection was not available for inspection.

Recommended action Require certificate-verified TLS in every pool and connection string, configure Supabase SSL enforcement, and reject startup configurations that disable verification. Do not use rejectUnauthorized false as a workaround.

Closure check Repeat the same pg_stat_ssl check from local development and both deployed apps. Require ssl true and a TLS version. Demonstrate that a non-TLS connection is refused after enforcement.

Evidence locations src/lib/db.ts:13 | packages/db/src/index.ts:21 | apps/admin/src/lib/admin-auth-database.ts:13

S02 Workplace roles can reach NFC cryptographic keys through the Data API

Assessment High severity. Confirmed hosted authorization mismatch.

Evidence Hosted clock_tags_select_manager permits owner, admin, manager and supervisor. Both browser roles have SELECT on ntag_aes_key_hex. The public identity helper accepts a raw Better Auth session token and checks expiry. The main getSession server function returns the complete session object, while the intended NFC setup UI separately requires the operator email allowlist or dedicated admin permissions.

Impact A legitimate workplace manager or supervisor with their own raw session token can satisfy the table policy and request their location's NFC AES key directly, bypassing the operator-only UI. An exposed key could support forged station scans for that location. This is not anonymous key disclosure or a demonstrated cross-workplace leak. No real NFC key was retrieved during the audit.

Recommended action Remove browser-role table SELECT on clock_tags and expose only a safe projection if direct client reads are needed. A column revoke alone does not override an existing table-level SELECT grant. Keep AES keys behind the dedicated clock_stations.manage server permission, with audited access.

Closure check Using a disposable workplace, confirm direct manager and supervisor requests for the AES column fail, operator setup still works, and ordinary scans continue through server-only RPC calls.

Evidence locations supabase/migrations/20260621000000_add_supervisor_role.sql:60 | supabase/migrations/20260409_better_auth_rls_policies.sql:40 | src/lib/auth-server.ts:8 | src/features/time-clock/server/admin-tags.ts:151

S03 Password reset leaves existing sessions valid

Assessment High severity. Confirmed code and installed library behavior.

Evidence The main auth configuration does not enable emailAndPassword.revokeSessionsOnPasswordReset. The installed Better Auth 1.7.12 resetCredentialPassword function deletes sessions only when that option is enabled. changeAccountPassword revokes other sessions, but the forgotten-password reset path simply calls auth.api.resetPassword.

Impact Someone holding a stolen session can remain signed in after the legitimate user resets their password to recover the account. The default session lifetime and five-minute cookie cache can extend the exposure.

Recommended action Enable revokeSessionsOnPasswordReset and make sensitive session checks authoritative. Review recovery behavior for passkeys and other account credentials. Keep reset tokens single use and short lived.

Closure check In an isolated account, retain a second browser session, reset the password, and verify the old session is rejected immediately by sensitive endpoints as well as after cache expiry. Test change-password separately.

Evidence locations src/lib/auth.ts:82 | src/features/account/server/actions.ts:43 | node_modules/better-auth/dist/utils/password.mjs:18

S04 Push subscription endpoints permit server side requests to arbitrary hosts

Assessment High severity. Confirmed request preparation and code path.

Evidence pushEndpointSchema accepts any URL. Verified users can store that endpoint, and push-service passes it to webPush.sendNotification without a host allowlist, private-address filter or explicit timeout. A local test with disposable cryptographic keys confirmed web-push prepares a POST to https://127.0.0.1:8443. No connection to an internal address was made.

Impact A verified account can plant an endpoint for a later notification, causing outbound HTTPS requests from the application server to attacker-selected or internal services. Reachability, valid TLS and network policy constrain exploitation. Slow endpoints can also occupy workers; saved subscriptions are not visibly capped per account.

Recommended action Accept HTTPS push endpoints only from documented supported push-provider hosts, reject credentials and unexpected ports, validate resolved addresses against private and reserved ranges, enforce egress restrictions and timeouts, and cap active subscriptions. Account for DNS rebinding.

Closure check Unit-test localhost, IPv4 and IPv6 private networks, encoded addresses, redirects, arbitrary domains and excessive subscription counts. Confirm real supported push providers still deliver. Do not test private destinations against production.

Evidence locations src/features/push-notifications/schemas/push-schemas.ts:5 | src/features/push-notifications/server/actions.ts:36 | src/features/push-notifications/server/push-service.ts:83

S05 Admin bootstrap can grant ownership without proving mailbox ownership

Assessment High severity. Conditional deployment risk.

Evidence The admin signup hook grants role owner when the submitted email matches ADMIN_BOOTSTRAP_EMAILS. Email verification is disabled and there is no visible one-time bootstrap lock or authenticated provisioning requirement. The local environment still has a bootstrap allowlist configured. Production allowlist values were not inspected.

Impact If a listed address has not already been registered, an outsider who knows it can claim the first account and become an admin owner. Existing registered addresses reduce this immediate opportunity; they do not make the reusable bootstrap mechanism safe. The admin app also has password-only login with no configured MFA or passkey plugin.

Recommended action Close bootstrap after initial provisioning and make it a one-time operator-controlled procedure. Require verified ownership before privilege assignment. Add strong MFA for the dedicated admin app and maintain a documented recovery process.

Closure check With an isolated database, attempt signup using an allowlisted but unregistered address and confirm no owner account is created without an authorized provisioning step. Verify the production bootstrap setting is empty or disabled.

Evidence locations apps/admin/src/lib/auth.ts:43 | apps/admin/src/lib/auth.ts:57 | apps/admin/src/lib/auth.ts:103

S06 Cookie caching delays session revocation checks

Assessment Medium severity. Confirmed configured behavior.

Evidence Main-app session.cookieCache is enabled for 300 seconds. readSessionFromRequestHeaders calls getSession without disableCookieCache. The installed library can return valid cached session data before consulting the database. The personal-data feature already requests disableCookieCache true, demonstrating an available authoritative pattern.

Impact Deleting a session centrally does not guarantee that every ordinary application endpoint immediately rejects the old cached session. Fresh membership checks reduce role-revocation risk but do not replace session revocation checks.

Recommended action Use authoritative session reads for billing, permission changes, exports, NFC setup and other sensitive actions. Decide and document the acceptable cache window for lower-risk page reads.

Closure check Revoke a session in a second browser and replay the first browser's cookies against sensitive actions before and after five minutes. Require immediate rejection for the sensitive actions.

Evidence locations src/lib/auth.ts:61 | src/lib/auth-session.server.ts:14 | src/features/personal-data/server/access.ts:10

S07 Legacy main app administration bypasses dedicated admin revocation

Assessment Medium severity. Confirmed alternate privileged access path.

Evidence The main /admin/clock-tags tools authorize a verified main-app account through ADMIN_EMAILS, independently of admin_private.admin_user, its disabled_at value and clock_stations.manage. The local environment configures ADMIN_EMAILS. The dedicated admin app correctly rechecks its own current membership.

Impact Disabling an operator in the dedicated admin app does not remove this separate route to read NFC keys and generate station setup material if that person's main-app email remains allowlisted. This requires an allowlisted main-app account, not an arbitrary user.

Recommended action Retire the legacy tools or route them through the same dedicated administrator permission and revocation model. Treat the main and admin origins as separate trust boundaries, with separate credentials and clear operator offboarding.

Closure check Disable a test operator in the dedicated admin app and confirm all privileged key-reading and setup endpoints, including legacy main-app paths, deny access.

Evidence locations src/features/time-clock/server/admin-tags.ts:151 | src/features/time-clock/server/admin-tags.ts:169 | apps/admin/src/features/auth/server/admin-session.ts:43

S08 Password recovery wrappers bypass auth router rate limiting

Assessment Medium severity. Confirmed missing application-level abuse control.

Evidence Public TanStack server functions delegate requestPasswordReset and resetPassword directly to auth.api methods. Those calls omit request context and the wrapper has no limiter. In the installed library, rate limiting is installed in the HTTP router onRequest path; direct API dispatch does not traverse it. Passing headers alone would not create that router-level protection.

Impact An unauthenticated caller can repeatedly invoke recovery work and send reset messages through a different surface than the rate-limited auth routes. CSRF checks are not request throttling. Missing request context also skips request-dependent callback-origin checks at the direct API boundary; no token-exfiltration exploit was demonstrated.

Recommended action Add bounded, shared IP and account-based rate limits to these server functions, constrain reset destinations to a fixed internal path, preserve request context where required, and return non-enumerating responses. Check equivalent wrappers around other auth operations.

Closure check With mocked mail, demonstrate limits apply to direct server-function calls, across workers and after repeated attempts. Confirm legitimate recovery still works and external callback destinations are rejected.

Evidence locations src/features/account/server-fns.ts:16 | src/features/account/server/actions.ts:32 | node_modules/better-auth/dist/api/index.mjs:172 | node_modules/better-auth/dist/api/to-auth-endpoints.mjs:44

S09 Passkey login redirects to an unchecked destination

Assessment Medium severity. Confirmed client authorization flow weakness.

Evidence The login route accepts any nonempty redirect search value. After explicit passkey success it assigns that value to window.location.href. That branch does not send the destination through Better Auth's callback validation. The password branch has an additional callbackURL validation opportunity and should not be assumed equally exploitable.

Impact A crafted login link can send a user to an attacker-controlled site after a genuine passkey login, supporting phishing. Same-origin session cookies are not automatically sent to that other site. The deployed CSP constrains javascript URL execution, so arbitrary script execution is not claimed.

Recommended action Validate redirect search values as same-origin internal paths at the route boundary, including protocol-relative URLs, backslashes and encoded forms. Reuse a small client-safe path validator.

Closure check Test https external destinations, double-slash paths, backslashes and javascript URLs. Require a safe dashboard fallback for both password and passkey login.

Evidence locations src/routes/login.tsx:23 | src/routes/login.tsx:99

S10 CSV exports preserve spreadsheet formulas in text fields

Assessment Medium severity. Confirmed export behavior.

Evidence serializeCsvCell quotes and escapes strings but does not neutralize formula prefixes. A local call with the harmless value =1+1 preserved it as a quoted CSV cell. Sage payroll exports use this serializer for employee names, payroll references, location and other text; current text schemas do not exclude formula prefixes.

Impact When a manager opens attacker-influenced exported text in a spreadsheet, it can be interpreted as a formula rather than data. Effects depend on the spreadsheet's protections and user actions. A direct Sage import may have different requirements.

Recommended action Provide spreadsheet-safe text serialization or a separate safe spreadsheet export, and preserve the payroll import format where necessary. Define handling for equals, plus, minus, at-sign, tab and carriage-return prefixes without corrupting legitimate numeric values.

Closure check Open controlled exports in the supported spreadsheet applications and confirm text stays literal. Run the existing Sage CSV import-compatibility tests after the change.

Evidence locations src/features/timesheets/utils/csv.ts:11 | src/features/payroll/adapters/sage-csv.ts:57 | src/features/company/schemas/company-schemas.ts:60

S11 Development shares hosted credentials and uses broad privileges

Assessment Medium severity. Confirmed local secret and privilege configuration.

Evidence The local and hosted environment files point to the same Supabase project and contain the same main auth secret. The tested database connection uses the RLS-bypassing postgres role. The server Supabase client uses a privileged secret key, and the inspected local Stripe key is an unrestricted secret rather than a restricted key. Actual deployed Stripe key permissions were not inspected.

Impact A compromised development machine or application path can affect the hosted employee database. Application authorization errors have a large blast radius because privileged clients bypass RLS. A shared auth signing secret also crosses the environment trust boundary.

Recommended action Separate development, staging and production databases, auth secrets, Stripe resources and email routing. Use restricted Stripe keys and narrower database roles where practical. Keep privileged Supabase clients server-only and retain explicit authorization at every entry point.

Closure check Record environment-to-project mappings without secret values, verify auth secrets differ, and test each narrowed role or restricted key against the full required workflow before rotating old credentials.

Evidence locations src/lib/supabase.server.ts:24 | src/lib/db.ts:14 | src/features/billing/server/stripe.ts:17

S12 Subscription risk events and event ordering need stronger handling

Assessment Medium severity. Confirmed payment control gaps.

Evidence The signed webhook handles subscription lifecycle and invoice events. Refund and dispute handlers only hold extra station orders. radar.early_fraud_warning.created has no branch. Subscription update payloads are applied directly without an event-age guard or retrieving the current Stripe state. The event log deduplicates processed IDs but does not atomically lease processing between concurrent requests.

Impact Refunded or disputed subscription payments have no explicit operator/risk policy in this handler. Delayed authentic updates can temporarily restore stale access state, and simultaneous deliveries can repeat downstream work. This is not proof of forged payment acceptance; signature verification and extra-station payment validation are strong.

Recommended action Define operator handling for subscription refunds, disputes and fraud warnings, then subscribe only to events acted upon. Reconcile authoritative current Stripe state, make processing claims atomic, and ensure downstream effects are idempotent. Do not blindly cancel subscriptions for every partial refund.

Closure check Replay duplicate, concurrent and deliberately out-of-order authentic fixtures. Check a cancellation cannot be undone by an older event. Test subscription refunds and disputes separately from one-off station orders, with no real charge required.

Evidence locations src/features/billing/server/webhook.ts:72 | src/features/billing/server/webhook.ts:81 | src/features/billing/server/webhook-events.ts:106 | src/features/billing/server/subscriptions.ts:89

S13 Dependency audit is no longer clean

Assessment Medium severity. Confirmed dependency advisories with limited runtime reachability.

Evidence The current npm audit returned 34 affected package entries: 11 high, 19 moderate and 4 low, with no critical. Production-classified dependencies returned 6 entries: 2 high and 4 low. These are dependency graph counts, not independent exploitable vulnerabilities. High production entries are shadcn and its nested Zod 3.25.76; application Zod is 4.6.7. Tooling-only high chains include Lighthouse and archive/proxy dependencies.

Impact Developer tooling and build environments carry known risks. shadcn is listed as a production dependency even though it is a generator, which increases installed attack surface. No attacker-reachable request path into the vulnerable nested Zod was established. Low deepmerge advisories also affect email tooling.

Recommended action Remove or move generator tooling out of production dependencies, update compatible transitive dependencies and inspect each advisory's actual reachability. Do not run npm audit fix force blindly. Recheck lockfile and runtime images after changes.

Closure check Repeat full and omit-dev audits, document justified exceptions, and verify the shipped server excludes generator, performance and archive tooling. Keep dependency scanning in the release process.

Evidence locations package.json:95 | package.json:123 | package-lock.json:19589

S14 Hosted billing reconciliation rejection returns a server error

Assessment Medium severity. Observed production failure with cause unverified.

Evidence An unauthenticated POST to https://rocketrota.com/api/billing/reconcile returned 500. The reviewed route should return 401 when a configured secret exists and the token is missing; getRequiredEnv runs before

that check. Missing deployment configuration is one plausible cause. The precise hosted revision and error logs were not available.

Impact The general billing recovery path cannot be treated as operationally verified. A configuration failure could prevent recovery from missed webhooks. The observed error is not proof that unauthenticated billing work ran or that authorization was bypassed.

Recommended action Inspect the deployed route and logs, configure a strong reconciliation secret and its authenticated schedule, return a controlled unavailable response for missing configuration, and add an actionable job-failure alert.

Closure check Require 401 for missing or invalid tokens and a successful authorized dry run or isolated test reconciliation. Confirm regular run records and failed-account alerts without exposing the bearer secret.

Evidence locations src/routes/api/billing/reconcile.ts:11 | docs/billing-reconciliation.md:45

S15 Database API grants exceed the server first design

Assessment Low severity. Confirmed unnecessary surface and defense in depth gaps.

Evidence The hosted advisor reports 49 anonymously discoverable and 49 authenticated GraphQL objects, plus 9 public SECURITY DEFINER functions callable by both roles. Their inspected bodies are session and role helpers, not unrestricted dump functions. Twenty RLS-without-policy notices mostly represent deliberate deny-by-default tables. Private schemas deny browser USAGE; 13 of 14 admin, 11 of 14 billing and the one push table lack RLS.

Impact These warnings do not establish anonymous row disclosure. Broad grants and unnecessary APIs increase the chance that a future policy error becomes exploitable, as S02 illustrates. Missing private-schema RLS reduces defense in depth but does not make those tables publicly accessible today.

Recommended action Inventory client-required objects and revoke unused grants from anon, authenticated and PUBLIC with dependency-aware testing. Disable unused GraphQL only after confirming no clients depend on it. Add appropriate private-schema RLS as defense in depth and restrict default privileges.

Closure check Re-run advisors and zero-row API checks. Distinguish discovery from data access, maintain safe projections, and ensure auth helper dependencies and service-role clock RPCs still work.

Evidence locations supabase/migrations/20260409_better_auth_rls_policies.sql:40 | supabase/migrations/20261023152746_restrict_sensitive_employee_data_api_access.sql:3

S16 Global headers replace the stricter privacy download policy

Assessment Low severity. Confirmed middleware behavior.

Evidence The privacy download explicitly requests a sandboxed default-src none CSP. After the handler returns, shared securityMiddleware sets every global security header on result.response, including CSP, replacing that stricter route policy with the ordinary app policy.

Impact The download loses its additional sandbox defense. HTML values are escaped and files are attachments, so a working script injection was not demonstrated. This is a defense-in-depth regression rather than proven data disclosure.

Recommended action Preserve stricter route-specific headers or provide a deliberate middleware override mechanism while retaining global baseline protections.

Closure check Test the final response headers of both HTML and JSON privacy downloads, not just the handler's header object. Require the intended sandbox policy to survive middleware composition.

Evidence locations src/features/personal-data/server/download.ts:13 | packages/shared/src/security/security-middleware.ts:29

Environment and TanStack boundary assessment

Private environment values belong on the server. Public configuration does not. The checked VITE_SUPABASE_KEY and VITE_SUPABASE_PUBLISHABLE_KEY are modern publishable keys, not privileged Supabase keys. A VAPID public key is also deliberately public. Keeping these public values does not violate the server-only secret boundary.

Configuration	Observed boundary and result
Database and auth secrets	Server-only env and pool/auth modules. No matching private values found in tracked files or fresh browser artifacts. TLS and environment isolation remain S01 and S11.
Supabase privileged keys	SUPABASE_SECRET_KEY is read by a server-only client. Browser Supabase configuration uses publishable keys. Dynamic NFC AES key disclosure is separately covered by S02.
Stripe and mail credentials	Private keys are read in server modules. No broad Vite define replacement or private VITE-prefixed variable was found. Live key permissions and actual hosting secret storage were not inspected.
Push configuration	VAPID_PRIVATE_KEY is read server-side. Public VAPID configuration is returned intentionally to a verified user. Destination validation remains S04.
Development switches	Verification bypass code is guarded by development mode. The local public bypass flag is true but its server flag is false; neither proves the deployed setting. Keep production bypasses disabled.

The exact-value scan examined 1686 tracked paths and 382 browser JS, CSS, HTML, JSON or map files; no configured private values matched, and no browser source maps were found. Relevant .env files are ignored and no commits were returned for the inspected credential-file paths. These checks do not prove the absence of unknown historical secrets, transformed values, runtime serialization leaks or values present only in Coolify.

A raw identifier search did find BETTER_AUTH_SECRET strings in bundled Better Auth environment helper code. These were property names and getters, not the secret values. They are not reported as a credential leak. Admin loadEnv with an empty prefix loads variables into server-side process.env; it does not by itself expose all variables to the browser.

TanStack guidance requires protection at the data/API boundary, not only beforeLoad route checks. The traced billing, company, manager-dashboard, timesheet, support and privacy entry points generally validate input and perform server-side session and permission checks. Both apps preserve the server-function CSRF middleware when defining a custom Start instance. The findings identify exceptions and alternate surfaces that UI guards do not close.

Continue using server-only import markers for env, database, auth and payment modules. Audit returned DTOs as well as imports: a server-only module can still intentionally or accidentally return sensitive data. Prefer minimal session DTOs over serializing the complete auth session, especially where the raw token can be reused by the database API.

Hosted database evidence

Control	Observed result
Public tables	55 of 55 have RLS enabled. No disabled-RLS public table was found.
Private schema access	anon and authenticated lack USAGE on account_private, admin_private, billing_private, privacy_private and push_private.
Sensitive base tables	Unauthenticated zero-row REST requests for user and employees returned 401. No customer rows were requested.
Other zero-row checks	account and clock_tags returned 200 for limit=0; time_entries returned 401. A 200 empty query proves endpoint reachability, not disclosure.

Control	Observed result
Identity and clock RPCs	A nonexistent synthetic session token returned null. claim_clock_scan_session EXECUTE is denied to anon and authenticated.
Advisor interpretation	20 RLS-without-policy notices; 49 GraphQL object notices for each browser role; 9 public definer-helper notices for each role. See S02 and S15.
Default privileges	New public tables, sequences and functions inherit broad browser-role grants for postgres and supabase_admin. Reduce future grants through tested migrations.
Schema version and products	83 migration versions match. No storage buckets or deployed Edge Functions were listed. This is not an organization-wide storage audit.

The public definer helpers inspected resolve session identity and roles rather than accepting arbitrary SQL or arbitrary user identity. Search paths are fixed, and anon lacks CREATE on public. Their callable status is an unnecessary API surface to review, not proof of anonymous privilege escalation. RLS without policies intentionally denies ordinary browser roles and should not be fixed by adding permissive policies.

Authentication and payment controls that passed review

- Main and admin use separate auth models and secrets in the inspected local configuration. Admin role and disabledAt are not accepted as generic profile input, and dedicated admin requests recheck active membership and permissions.
- Main workspace operations derive the user from the server session and check current organization/location permissions in the traced paths. Billing initiation and portal access are restricted to the owner. Privacy requests bypass the cookie cache and require verified email.
- Better Auth supplies password hashing, secure-cookie behavior and origin protections. Current configuration does not disable CSRF protections. Runtime cookie attributes and authenticated cross-origin rejection still require an actual controlled login test.
- The custom Stripe webhook verifies the raw request signature. An unsigned live request returned 400. Local code records processed event IDs and implements subscription, invoice, failed-payment and recovery paths.
- Extra-station fulfillment checks immutable order identity, customer, exact price, quantity, currency, amount and a settled PaymentIntent. Refund/dispute holds exist for those one-off orders. Test and zero-value invoices do not release included hardware.
- The main and admin sites return per-request nonce CSP, DENY framing, nosniff, referrer policy and private no-store caching on sampled HTML/auth responses. The public .env probe returned 404. No production .env content was printed.
- Service-worker code uses the network for private document navigation and limits ordinary caching to assets. Privacy HTML export escapes record values and delivers attachments; preserve its stricter CSP as S16 describes.

Payment configuration still needs account-level verification: live webhook event subscriptions, endpoint version and signing secret, restricted key permissions, recovery settings, risk alerts and tax registrations where applicable. Checkout enables automatic tax, but code alone cannot establish the account registration or correct tax treatment. No live charge or refund was created for this audit.

Verification record

Check	Result
Selected main security tests	8 files and 44 tests passed. Auth, signup gate, station payments, station alerts, security headers and privacy server functions included.
Admin test suite	12 files and 49 tests passed using the bundle config loader and a single thread worker.

Check	Result
Additional focused rerun	6 main files and 34 tests passed; these overlap the earlier 44 and are not counted twice.
Database integration tests	2 files and 9 tests skipped because DATABASE_URL was not supplied to Vitest. No rollback fixtures were run against production.
Main production build	Passed on retry outside the sandbox. The first attempt failed with a sandbox EPERM on the PDF worker realpath, not a demonstrated code compilation error.
Admin production build	Passed. Both fresh public outputs were included in the private-value scan.
Full npm audit	34 affected package entries: 11 high, 19 moderate, 4 low, 0 critical. No dependencies were changed.
Production npm audit	6 affected package entries: 2 high and 4 low. Nested tooling reachability needs review; application Zod 4.6.7 is not the flagged copy.
Local safe reproductions	Loopback push request preparation accepted; harmless CSV formula preserved. No internal request or real spreadsheet formula execution occurred.
Live rejection checks	Both unauthenticated update-user probes returned 401; unsigned webhook 400; reconciliation without bearer token 500. The 401 results do not prove authenticated CSRF behavior.

Useful saved evidence is in tmp/security-audit-2026-10-30: environment-summary.json, runtime-summary.json, hosted-key-grants.json, database-functions.json, database-policies.json, hosted-migrations.json, npm-audit.json, npm-audit-production.json, reproductions.json, rejection-checks.json and build logs. All local private values remained in memory during scans and are omitted from these summaries. The report does not rely on the older audit claiming zero dependency advisories.

Operational checks not established by this audit

These are verification gaps, not confirmed absent controls. Close them with concrete evidence before launch; passing functional workbook scenarios alone does not establish each infrastructure control.

- Backup and restore: document retention, access, off-site recovery and acceptable data loss, then restore into an isolated database. The hosted organization is currently Free; do not assume paid-plan accessible daily backups. Include any future object-storage files separately.
- Hosting and operator access: inspect Coolify/VPS secrets, database network restrictions, deployed TLS settings, proxy IP handling, operator MFA, patching, SSH access and account recovery. These control panels were not accessed.
- Jobs and alerts: verify clock processing, privacy purge/email and station recovery schedules survive restart; verify general billing reconciliation and actionable failure notifications. Source task declarations do not prove execution.
- Authorization testing: use two disposable workplaces and all roles to test direct requests, role changes, offboarding, stale sessions, exports and privileged tag tools. No authenticated cross-tenant penetration test or real user session manipulation was performed.
- Abuse and availability: establish proxy/request body limits, shared rate limits, timeouts and resource quotas. Test them in staging rather than load-testing the employee database or production email service.
- Release controls: verify the deployed revision, rerun clean builds and scans in CI, retain a known-good artifact and migration recovery plan, and verify full regression coverage. This audit did not run every application test or certify the live Stripe account.

Additional hardening: restrict production trustedOrigins to intended HTTPS origins rather than keeping localhost entries; minimize auth-session DTOs; review password policy and breached-password screening; and define protected operational-log retention. These should be prioritized after the concrete high findings, without replacing the existing protections.

Remediation and retest order

Order	Work	Required closure evidence
1	S02 and S07 NFC permissions	Manager and supervisor direct key reads denied; dedicated operator setup works; disabling an operator closes every privileged path.
2	S01 and S11 transport and secrets	Verified TLS from each client/app, independent environment credentials and tested least-privilege access.
3	S03 S05 S06 S08 auth	Recovery revokes sessions; sensitive actions reject old sessions; bootstrap closed; shared abuse limits and strong admin authentication tested.
4	S04 S09 S10 untrusted inputs	Push private destinations blocked, safe login destinations enforced, and safe spreadsheet export demonstrated.
5	S12 S13 S14 release and billing	Risk/replay fixtures pass, dependency exceptions documented, authorized reconciliation and failure alerts verified.
6	S15 S16 defense in depth	Least necessary API grants, tested defaults and private-schema protections, and final privacy-download CSP confirmed.

This engagement produced findings and evidence only. Fixes should be implemented as focused changes with regression tests and an isolated retest before production configuration or schema changes. Re-run this audit against the release candidate after remediation.

Primary references

TanStack Start environment variables
https://tanstack.com/start/latest/docs/framework/react/guide/environment-variables
TanStack Start import protection
https://tanstack.com/start/latest/docs/framework/react/guide/import-protection
TanStack Start server functions
https://tanstack.com/start/latest/docs/framework/react/guide/server-functions
TanStack Start authentication
https://tanstack.com/start/latest/docs/framework/react/guide/authentication
Better Auth security
https://better-auth.com/docs/reference/security
Better Auth session management
https://better-auth.com/docs/concepts/session-management
Better Auth email and password
https://better-auth.com/docs/authentication/email-password
Supabase securing the Data API
https://supabase.com/docs/guides/api/securing-your-api
Supabase SSL enforcement
https://supabase.com/docs/guides/platform/ssl-enforcement
Supabase database backups
https://supabase.com/docs/guides/platform/backups
Stripe subscription webhooks and risk events
https://docs.stripe.com/billing/subscriptions/webhooks
Zod constrained collections advisory
https://github.com/advisories/GHSA-mrhx-c7xc-m3w5
deepmerge prototype poisoning advisory
https://github.com/advisories/GHSA-gcqx-w48m-w2h4